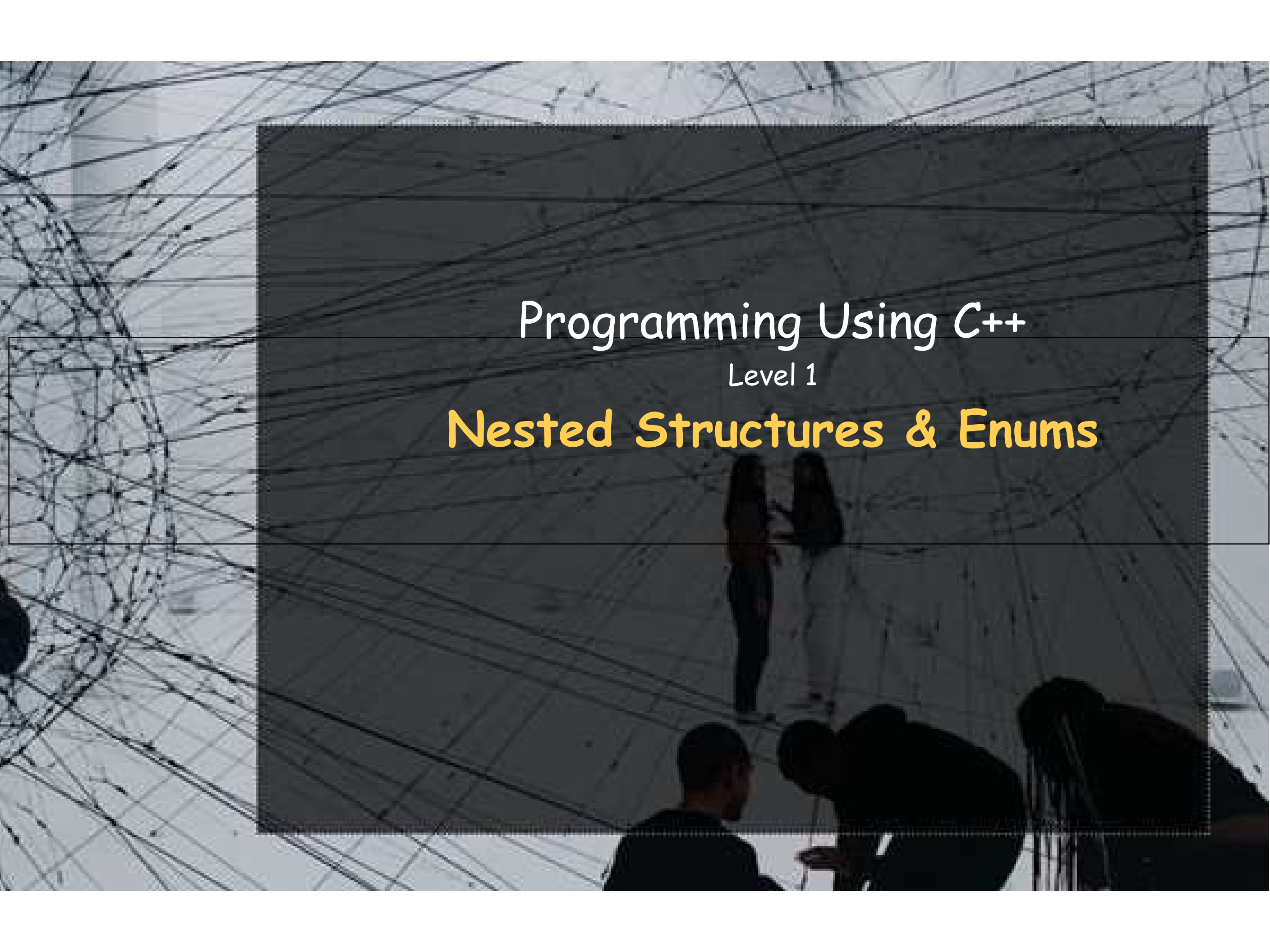


Programming Using C++ Level 1

Nested Structures & Enums



Programming Using C++

Level 1

Nested Structures & Enums

🔍 Definition

- ◆ A nested structure is a structure that contains another structure as one of its members.
- ◆ An enum (enumeration) is a user-defined type that gives names to a set of related constant values.
- ◆ This program combines both concepts to organize a person's personal and contact information.

Explanation

- ◆ The program defines three enums:
 - ``enColor`` → Red, Green, Blue
 - ``enGender`` → Male, Female
 - ``enMaritalStatus`` → Single, Married
- ◆ It also defines three structures:
 - ``stAddress``
 - ▨ Stores street, post number, city, and country.
 - ``stContactInfo``
 - ▨ Stores phone, mail, and an ``stAddress``.
 - ``stName``
 - ▨ Stores first name, last name, and ``stContactInfo``.
- ◆ This creates multiple levels of nested structures.

☰ Steps in main()

1☐ Define the enums

 ◆ Create `enColor`, `enGender`, and `enMaritalStatus` with predefined values.

2☐ Define the address structure

 ◆ Create `stAddress` to store address information.

3☐ Define the contact information structure

 ◆ Create `stContactInfo` and include `stAddress` inside it.

4☐ Define the name structure

 ◆ Create `stName` and include `stContactInfo` inside it.

5☐ Create the variables

 ◆ Create a `stName` object called `Name`.

 ◆ Create variables for age, salary, gender, marital status, and color.

6☐ Read the user's information

 ◆ The program reads the person's name, age, phone, email, address, salary, gender, marital status, and favorite color.

7☐ Access nested structure members

 ◆ The dot `.` operator is used to access members at different levels.

☐ Steps in main()

8☐ Use enum values

- ◆ The program uses enum values such as `Male`, `Female`, `Single`, `Married`, `Red`, `Green`, and `Blue`.
- ◆ By default, enum values start from `0`.

9☐ Display the information

- ◆ The program displays the stored information and uses the enum values to display meaningful text instead of only numbers.

💡 Example

Name.ContactInfo.Address.City

⚡ This accesses:

▤ `Name`

▨ `ContactInfo`

▨ `Address`

▨ `City`

➡ Result

⚡ The program reaches the `City` member stored inside the nested structures.

✈ Note

◆ In this program:

■ `Male = 0`, `Female = 1`

■ `Single = 0`, `Married = 1`

■ `Red = 0`, `Green = 1`, `Blue = 2`

◆ These values are assigned automatically because no values were explicitly specified.

◆ `reinterpret\_cast<int&>(Gender)` is used here to read the numeric input directly into the enum variable.

◆ This is a low-level technique and is not usually the preferred way to read enum values. It is mainly being used in this example to allow the user to enter the enum's underlying number.

⊘ Warning

◆ Enum values should be used carefully because entering a number outside the defined enum values can produce a value that does not correspond to one of the named enum members.

Comparison

◆ Enum values should be used carefully because entering a number outside the defined enum values can produce a value that does not correspond to one of the named enum members.

✦✦ Summary

- ✓ Nested structures allow one structure to contain another structure.
- ✓ Enums provide named values for a fixed set of choices.
- ✓ Nested structures and enums make complex data more organized and easier to understand.

Thank you 😊